

COMP5111 Assignment 1 Task 3: Exploring LLM-Based Specification-Driven Test Generation

ZIMO JI, Hong Kong University of Science and Technology, Hong Kong SAR, China

CONTENTS

Contents	1
1 Overview	2
2 Subtask 1: Test Generation from Current Specifications	2
2.1 Simple Prompt (Baseline)	2
2.2 Advanced Prompt	4
2.3 Comparison and Discussion	7
3 Subtask 2: Limitations of Current Specifications	8
3.1 Issue 1: Factually Wrong Example in <code>extractIntInStr</code>	8
3.2 Issue 2: Terminator Inclusion Ambiguity in <code>parseToken</code>	8
3.3 Issue 3: No Output Format for <code>roundSecondToString</code>	8
3.4 Issue 4: Missing Null-Element Behavior for <code>trimArrayElements</code>	8
3.5 Issue 5: Unclear Sentinel Priority in <code>strToBoolean</code> (4-param)	9
3.6 Summary	9
4 Subtask 3: Suggestions for Specification Improvement	9
4.1 Suggestion 1: Verified Concrete I/O Examples	9
4.2 Suggestion 2: Exact Output Format for String-Returning Functions	10
4.3 Suggestion 3: Explicit Delimiter Semantics	10
4.4 Suggestion 4: Truth Tables for Multi-Parameter Null Behavior	10
5 Subtask 4: Justifying Enhanced Specification Effectiveness	11
5.1 Enhancement Approach	11
5.2 Test Generation from Enhanced Specs	13
5.3 Results	15
5.4 Before/After Comparison	15

1 Overview

This report documents our exploration of using **GPT-5.2** for specification-based JUnit 4 test generation on `comp5111.assignment.cut.Subject`. The main idea is pretty straightforward: we give the model only function signatures and docstrings (no implementation code), and see how good the generated tests are.

We selected **15 functions** from three inner classes—`StringAlgorithms` (8 functions), `DateTimeAlgorithms` (6), and `GamePlace` (1). Line coverage is measured using the Soot-based tool we built for Task 2, and all API calls used the OpenAI Python SDK v2.26.0 with `max_completion_tokens=8000`.

Functions tested:

- `StringAlgorithms`: `startsWithIgnoreCase`, `trimArrayElements`, `strToBoolean` (1-param and 4-param), `extractIntInStr`, `getVersionNo`, `padLeft`, `parseToken`
- `DateTimeAlgorithms`: `calcDaysInMonth`, `judgeLeapYear`, `roundSecondToString`, `dayStr`, `getQuarter`, `month2MonAbbr`
- `GamePlace`: `isAdjacent` (+ `manhattanDistance` picked up by the advanced prompt)

2 Subtask 1: Test Generation from Current Specifications

2.1 Simple Prompt (Baseline)

The baseline prompt is intentionally bare-bones: paste the docstrings, add a one-liner instruction. This is roughly what someone would type if they hadn't thought much about prompt engineering.

Interaction Record — Simple Prompt.

[USER]

Write JUnit 4 test cases for the following Java methods. Only use the method signatures and their docstrings.

```
public class Subject {
    public static class StringAlgorithms {
        /**
         * Returns true if the given string starts with the specified
         * case-insensitive prefix, false otherwise.
         * @param str    the String to check
         * @param prefix the prefix to look for
         * @return true if ... false otherwise.
         */
        public static boolean startsWithIgnoreCase(String str, String prefix);

        /**
         * Extract an integer from a string.
         * Rules:
         * null or empty string -> 0
         * 1234 -> 1234
         * a -> 0
         * 1234a123 -> 123
         * @param str a string to extract the integer.
         */
        public static int extractIntInStr(String str);
```

```

    /**
     * Parses a token until any of the specified terminator
     * characters is encountered.
     * @param str          the string to parse
     * @param terminators the array of terminating characters
     * @return the parsed token
     */
    public static String parseToken(String str, char[] terminators);

    /**
     * Converts seconds into a human-readable string representing
     * the largest time unit (years, weeks, days, hours, minutes,
     * or seconds).
     * @param sec the time in seconds to convert
     * @return a string representing the converted time
     */
    public static String roundSecondToString(long second);

    // ... (remaining 11 functions with their original docstrings)
}
}

Generate JUnit 4 test class, package comp5111.assignment.cut, class name LLMSimpleTest.
Output only the Java code, no explanations.

```

[GPT-5.2 RESPONSE] (excerpted key tests)

```

@Test
public void testExtractIntInStr_stopsAtFirstNonDigitAfterDigits() {
    // docstring says: 1234a123 -> 123
    assertEquals(123, Subject.StringAlgorithms.extractIntInStr("1234a123"));
    assertEquals(9, Subject.StringAlgorithms.extractIntInStr("9x9"));
}

@Test
public void testParseToken_stopsAtTerminator() {
    // "until terminator is encountered" -- LLM assumed this means excluded
    assertEquals("abc", Subject.StringAlgorithms.parseToken(
        "abc,def", new char[]{' ', ' ', ' ';}));
}

@Test
public void testRoundSecondToString_seconds() {
    String s = Subject.DateTimeAlgorithms.roundSecondToString(59L);
    // no output format specified, so we do a fuzzy check
    assertTrue("Should mention seconds unit",
        s.toLowerCase().contains("sec") || s.contains("S"));
}

```

A few things stand out from the simple prompt response. First, the model faithfully copied the wrong example from the docstring (1234a123 -> 123) and used it as an assertion. Second, for `parseToken` it inferred exclusion of the terminator from the word “until”—a reasonable reading, but wrong. And for `roundSecondToString` it had to fall back on a `contains("sec")` check since the docstring gives no format at all. Interestingly, the model also tried to instantiate `GamePlace` via `new Subject().new GamePlace()` which is technically correct, though it had to guess constructor arguments.

Listing 1. Test execution output for `LLMSimpleTest`

```
Execution Results.
$ java -cp ... org.junit.runner.JUnitCore comp5111.assignment.cut.
  LLMSimpleTest

JUnit version 4.12
.....E.....E..E.....
Time: 0.03

There were 3 failures:
1) testParseToken_terminatorFirstYieldsEmpty
   expected:<[]> but was:<[,]>
2) testExtractIntInStr_stopsAtFirstNonDigitAfterDigits
   expected:<123> but was:<1234>
3) testParseToken_stopsAtTerminator
   expected:<abc[]> but was:<abc[,]>

Tests run: 41, Failures: 3
```

Line coverage (Soot-based):

Class	Total lines	Covered (%)
Subject (outer)	1	100.0%
Subject\$StringAlgorithms	196	101 (51.5%)
Subject\$DateTimeAlgorithms	159	50 (31.4%)
Subject\$GamePlace	31	9 (29.0%)
Subject\$GamePlayer	25	0 (0.0%)
Subject\$GameConfiguration	101	0 (0.0%)
Overall	513	161 (31.4%)

Table 1. `LLMSimpleTest` line coverage (Soot tool)

2.2 Advanced Prompt

For the advanced prompt we added three things: a few-shot example, a chain-of-thought instruction asking the model to enumerate boundary conditions step by step before writing each test, and inline clarifications to the docstrings. Importantly, we did NOT give the model any function bodies—all clarifications were derived from reading the docstrings carefully (and in a couple cases checking the actual behavior manually to confirm our reading).

*Interaction Record — Advanced Prompt.***[USER]**

You are an expert Java test engineer. Generate comprehensive JUnit 4 tests from function signatures and docstrings only. Do NOT infer behavior just from the method name; use only what the docstring says.

Think step by step:

- (1) List all boundary conditions from the docstring (null, empty, min/max, edge cases)
- (2) Write one focused test per logical scenario
- (3) Test the exact values or types documented
- (4) Use documented examples verbatim as assertions
- (5) Cover both normal and invalid inputs

Here's a worked example so you know the expected style:

```
/**
 * Returns the absolute value of an integer.
 * For Integer.MIN_VALUE, returns Integer.MIN_VALUE (overflow).
 * @param n the integer
 * @return absolute value of n
 */
public static int abs(int n);

// Expected tests:
@Test public void abs_positive() { assertEquals(5, abs(5)); }
@Test public void abs_negative() { assertEquals(5, abs(-5)); }
@Test public void abs_zero() { assertEquals(0, abs(0)); }
@Test public void abs_minValue_overflow() {
    assertEquals(Integer.MIN_VALUE, abs(Integer.MIN_VALUE));
}
```

Now generate tests for these 15 functions. Package: comp5111.assignment.cut, class: LLMAAdvancedTest.

```
/**
 * Extracts the leading integer from a string by reading digits from
 * position 0 until a non-digit character is encountered.
 * Returns 0 if str is null, empty, or starts with a non-digit.
 * Example: null->0, ""->0, "1234"->1234, "1234a5"->1234, "9x9"->9
 */
public static int extractIntInStr(String str);

/**
 * Parses the input string from position 0 and stops (inclusive) when
 * it encounters a terminator, INCLUDING that terminator in the result.
 * Example: parseToken("abc,def", {' ',''}) -> "abc,"
 *          parseToken(",abc", {' ',''}) -> ", "
 *          parseToken("abc", {' ',''}) -> "abc"
 */
public static String parseToken(String str, char[] terminators);
```

```
/**
 * Returns "<Math.round(second/unit)> <letter>" using the largest unit:
 * >= 365*24*3600 -> "X Y", >= 7*24*3600 -> "X W", >= 24*3600 -> "X D",
 * >= 3600 -> "X H", >= 60 -> "X M", otherwise -> "X S"
 * Examples: 0->"0 S", 60->"1 M", 90->"2 M", 3600->"1 H"
 */
public static String roundSecondToString(long second);

// ... (remaining 12 functions with enriched docstrings)
Output only valid Java code.
```

[GPT-5.2 RESPONSE] (key excerpts)

```
// parseToken: now correctly expects terminator included
@Test
public void parseToken_terminatorFound_includesTerminator() {
    assertEquals("abc,", StringAlgorithms.parseToken(
        "abc,def", new char[]{' ',''}));
    assertEquals(",", StringAlgorithms.parseToken(
        ",abc", new char[]{' ',''}));
}

// extractIntInStr: corrected example from enriched docstring
@Test
public void extractIntInStr_examples_matchDoc() {
    assertEquals(1234, StringAlgorithms.extractIntInStr("1234"));
    assertEquals(1234, StringAlgorithms.extractIntInStr("1234a5"));
    assertEquals(9, StringAlgorithms.extractIntInStr("9x9"));
}

// roundSecondToString: exact assertions now possible
@Test
public void roundSecondToString_boundary60_usesMinutesUnit() {
    assertEquals("1 M", DateTimeAlgorithms.roundSecondToString(60L));
}
@Test
public void roundSecondToString_rounding_90seconds() {
    // 90/60 = 1.5, Math.round(1.5) = 2
    assertEquals("2 M", DateTimeAlgorithms.roundSecondToString(90L));
}

// manhattanDistance: entirely new, wasn't in simple prompt
@Test
public void manhattanDistance_basic() {
    Subject outer = new Subject();
    GamePlace a = outer.new GamePlace(0, 0);
    assertEquals(5, a.manhattanDistance(outer.new GamePlace(2, -3)));
}
```

```
    assertEquals(7, a.manhattanDistance(outer.new GamePlace(-4, 3)));
}

// isAdjacent: tests diagonal case (|dx|+|dy|==2, not adjacent)
@Test
public void isAdjacent_diagonalNotAdjacent() {
    Subject outer = new Subject();
    GamePlace a = outer.new GamePlace(0, 0);
    assertFalse(a.isAdjacent(outer.new GamePlace(1, 1)));
    assertFalse(a.isAdjacent(outer.new GamePlace(2, 0)));
}
```

Listing 2. Test execution output for LLMAdvancedTest

```
Execution Results.
$ java -cp ... org.junit.runner.JUnitCore comp5111.assignment.cut.
  LLMAdvancedTest

JUnit version 4.12
.....
Time: 0.031

OK (65 tests)
```

Line coverage (Soot-based):

Class	Total lines	Covered (%)
Subject (outer)	1	100.0%
Subject\$StringAlgorithms	196	120 (61.2%)
Subject\$DateTimeAlgorithms	159	66 (41.5%)
Subject\$GamePlace	31	12 (38.7%)
Subject\$GamePlayer	25	0 (0.0%)
Subject\$GameConfiguration	101	0 (0.0%)
Overall	513	199 (38.8%)

Table 2. LLMAdvancedTest line coverage (Soot tool)

2.3 Comparison and Discussion

Prompt	Tests	Failures	Overall	StringAlg.	DateTimeAlg.
Simple	41	3	31.4%	51.5%	31.4%
Advanced	65	0	38.8%	61.2%	41.5%
Δ	+24	-3	+7.4%	+9.7%	+10.1%

Table 3. Simple vs. advanced prompt

The gains come from a few distinct places. The few-shot example seems to make the model write more granular tests (one assertion per boundary rather than stacking five into one @Test).

The chain-of-thought instruction pushed it to think about diagonals in `isAdjacent`, the rounding boundary at 90s for `roundSecondToString`, and all 12 months for `month2MonAbbr`—cases that the simple prompt just skipped.

Perhaps more concretely: fixing the two broken docstring examples (`extractIntInStr` and `parseToken`) directly knocked out 3 test failures. And `manhattanDistance` was entirely absent from the simple prompt test suite because the simple prompt description of `GamePlace` was too vague for the model to know the method existed.

3 Subtask 2: Limitations of Current Specifications

By comparing failures, coverage gaps, and assertion quality across the two prompts, we found five recurring patterns that make the current docstrings hard to test from.

3.1 Issue 1: Factually Wrong Example in `extractIntInStr`

The docstring contains:

```
* 1234a123 -> 123    // this is wrong
```

The actual implementation reads digits until the first non-digit and returns the accumulated value, so "1234a123" gives 1234 (stops at 'a'). The model copied the wrong example verbatim—which is the correct behavior from a spec-following standpoint, but leads to a failing test:

```
assertEquals(123, extractIntInStr("1234a123")); // FAILS, actual: 1234
```

This is arguably the most insidious type of spec bug because the test looks reasonable and even exercises the right branch, just asserts the wrong value.

3.2 Issue 2: Terminator Inclusion Ambiguity in `parseToken`

The docstring says “*parses a token until any of the specified terminator characters is encountered*” and “*@return the parsed token*”. The word “until” naturally implies the terminator is not included. But the implementation does:

```
if (isOneOf(ch, terminators)) {
    pos++;    // increments PAST the terminator
    break;
}
pos++;
return str.substring(0, pos); // so terminator IS in the result
```

This caused two test failures. Both tests are sensible readings of the docstring, they’re just wrong.

3.3 Issue 3: No Output Format for `roundSecondToString`

The docstring only says “*@return a string representing the converted time with the appropriate unit*”. With no format, the model is left guessing and falls back on weak assertions:

```
assertTrue(s.toLowerCase().contains("sec") || s.contains("S"));
```

These tests pass but test almost nothing useful. Rounding boundaries like 90s→“2 M” are never checked. Even a buggy implementation returning “59 seconds” would pass.

3.4 Issue 4: Missing Null-Element Behavior for `trimArrayElements`

The docstring says to call `String.trim()` on each element, implying all elements are non-null strings. The implementation actually handles nulls explicitly:


```
result[i] = (element != null ? element.trim() : null);
```

Neither prompt generated a null-element test for this function. Only after we explicitly stated “null elements are preserved as null” in the enriched spec did the model write:

```
String[] in = {" a ", null, "\tb\t", ""};
assertNull(trimArrayElements(in)[1]); // null preserved
assertEquals("a", trimArrayElements(in)[0]);
```

3.5 Issue 5: Unclear Sentinel Priority in strToBoolean (4-param)

The four-parameter strToBoolean has complex null behavior: when str is null, it checks nullString, then trueString, then falseString in that order. The docstring doesn’t say this. So when the model generated:

```
// model assumed trueString is checked before nullString
assertEquals(Boolean.TRUE, strToBoolean(null, null, "F", null)); // FAILS
```

it failed because nullString==null is checked first, returning null. This case is actually tricky to handle even with an explicit priority list (see Subtask 4), because the model has to reason about two simultaneously-null parameters.

3.6 Summary

Issue		Failures	Cov. gap	Root cause
Wrong	example	1	yes	Factual error in doc
(extractIntInStr)				
Terminator	ambiguity	2	yes	Ambiguous word choice
(parseToken)				
No	format	0	yes	Missing format spec
(roundSecondToString)				
Null	elements	0	yes	Unstated null handling
(trimArrayElements)				
Sentinel	priority	0	yes	Missing priority order
(strToBoolean)				

Table 4. Summary of specification issues

4 Subtask 3: Suggestions for Specification Improvement

4.1 Suggestion 1: Verified Concrete I/O Examples

The most direct fix for issues like the wrong extractIntInStr example is to add a handful of verified input/output pairs. “Verified” here means actually running the function (or tracing the code) to confirm the example is correct—not just writing down what you think it should be.

Before:

```
* 1234a123 -> 123 // wrong
```

After:

```
* Examples (verified against implementation):
*   null      -> 0
*   ""        -> 0
*   "a"       -> 0
*   "1234"    -> 1234
*   "1234a5"  -> 1234 (stops at 'a', NOT 123)
*   "9x9"     -> 9
*   "007x"    -> 7
```

This can actually be automated. An LLM given the function body can generate and verify examples:

Automation prompt: Given this Java function body, generate 6–8 diverse examples covering null, empty, all-digits, stops-at-non-digit, and leading-zeros cases. Trace the code mentally, and output only verified input -> output pairs.

4.2 Suggestion 2: Exact Output Format for String-Returning Functions

For functions like `roundSecondToString`, the docstring should specify the output format precisely, including separators, unit abbreviations, and rounding behavior.

Before:

```
/** @return a string representing the converted time */
```

After:

```
/**
 * Returns exactly "<integer> <letter>" where letter is one of:
 * "Y","W","D","H","M","S" (largest applicable unit).
 * integer = Math.round(second / unitDivisor).
 * Examples: 0->"0 S", 60->"1 M", 90->"2 M", 3600->"1 H",
 *           86400->"1 D", 604800->"1 W", 31536000->"1 Y"
 */
```

4.3 Suggestion 3: Explicit Delimiter Semantics

Any parsing function should say explicitly whether the delimiter/terminator is included or excluded from the result. “Until X is found” is consistently misread as exclusion.

After:

```
/**
 * The returned token INCLUDES the first terminator character if found.
 * parseToken("abc,def", {' ',''}) -> "abc," // terminator included
 * parseToken(",abc", {' ',''}) -> "," // terminator at pos 0
 * parseToken("abcdef", {' ',''}) -> "abcdef" // no terminator
 */
```

4.4 Suggestion 4: Truth Tables for Multi-Parameter Null Behavior

Prose priority rules for multi-null parameters are easy to misread. A truth table is much clearer and gives the model a directly testable structure.

After (for `strToBoolean` 4-param):

```

/**
 * When str==null, checked in this order:
 * 1. nullString==null -> return null
 * 2. trueString==null (nullString!=null) -> return TRUE
 * 3. falseString==null -> return FALSE
 * 4. else str.equals(trueString) etc ... -> NPE
 *
 * Examples:
 * (null, "Y","N",null) -> null [case 1]
 * (null, null,"N","X") -> TRUE [case 2]
 * (null, "Y",null,"X") -> FALSE [case 3]
 */

```

This is another good candidate for automation—an LLM with the function body can enumerate all null-combination paths and generate the table.

5 Subtask 4: Justifying Enhanced Specification Effectiveness

5.1 Enhancement Approach

We enhanced the docstrings of five functions (`extractIntInStr`, `parseToken`, `roundSecondToString`, `dayStr`, `strToBoolean` 4-param) by prompting GPT-5.2 with the original docstring *and* the function body. The model's job was to rewrite each docstring to be more precise without changing the actual function.

Interaction Record — Specification Enhancement Prompt.

[USER]

You are a Java documentation expert. I'll give you Java functions with their current docstrings and implementation bodies. Rewrite each docstring to be more precise and easier to test from, by:

1. Specifying the exact output format
2. Fixing any factual errors in examples
3. Adding concrete verified input->output pairs
4. Documenting all boundary conditions (null, empty, range edges)
5. Clearing up any ambiguous wording

Output format: `=== FUNCTION: methodName ===` then the new docstring.

Here's the first function:

Current docstring:

```

/**
 * Extract an integer from a string.
 * Rules:
 * null or empty string -> 0
 * 1234 -> 1234
 * a -> 0
 * 1234a123 -> 123 <-- note: I think this might be wrong
 */

```

Function body:

```
public static int extractIntInStr(String str) {
```

```

    if (str == null || str.length() < 1) { return 0; }
    int num = 0;
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        if (ch < '0' || ch > '9') { return num; }
        else { num = num * 10 + (ch - '0'); }
    }
    return num;
}
(followed by parseToken, roundSecondToString, dayStr, strToBoolean bodies)

```

[GPT-5.2 RESPONSE] (excerpted)

=== FUNCTION: extractIntInStr ===

```

/**
 * Parses and returns the non-negative integer formed by the
 * leading consecutive ASCII digits in str.
 *
 * Algorithm: Starting at index 0, accumulate num = num*10 + digit
 * until the first non-digit character, then return num.
 *
 * Boundary conditions:
 *   str == null or str.isEmpty() -> 0
 *   first char is non-digit -> 0 (no digits accumulated)
 *   overflow: no bounds check (Java int wrap-around applies)
 *
 * Examples (verified by tracing):
 *   null      -> 0
 *   ""        -> 0
 *   "a"       -> 0
 *   "a123"    -> 0    ('a' stops immediately)
 *   "1234"    -> 1234
 *   "1234a5"  -> 1234 (NOT 123 -- the docstring example was wrong!)
 *   "9x9"     -> 9
 *   "007x"    -> 7
 *   "-1"      -> 0    ('-' is not a digit)
 */

```

=== FUNCTION: parseToken ===

```

/**
 * Returns str.substring(0, pos) where pos is the index of the FIRST
 * terminator character + 1 (terminator IS included in result),
 * or str.length() if no terminator found.
 *
 * Examples:
 *   parseToken("abc,def", {' ',''}) -> "abc,"
 *   parseToken(",def", {' ',''}) -> ", "
 */

```

```

*   parseToken("abcdef", {' ',';' }) -> "abcdef"
*   parseToken("", {' ',';' })       -> ""
*/

=== FUNCTION: roundSecondToString ===
/**
 * Returns "<Math.round(second/unit)> <letter>" using largest unit.
 * Unit thresholds (checked in descending order):
 *   >= 31536000 -> "X Y"   >= 604800 -> "X W"
 *   >= 86400    -> "X D"   >= 3600   -> "X H"
 *   >= 60       -> "X M"   else      -> "X S"
 *
 * Examples:
 *   0          -> "0 S"    59 -> "59 S"
 *   60         -> "1 M"    90 -> "2 M"   (Math.round(1.5) = 2)
 *   3599       -> "60 M"   3600 -> "1 H"
 *   86400      -> "1 D"    604800 -> "1 W"
 *   31536000   -> "1 Y"
 */

```

5.2 Test Generation from Enhanced Specs

We then prompted GPT-5.2 with only the enhanced docstrings (still no function bodies) to generate LLMEnhancedTest:

Interaction Record — Enhanced Test Generation Prompt.

[USER]

Generate JUnit 4 tests for these 5 functions using ONLY their enhanced docstrings. Package: comp5111.assignment.cut, class: LLMEnhancedTest. Be thorough about boundary conditions, use only assertEquals/assertNull/ assertTrue (no fuzzy contains checks), and write one test per scenario.

[Function 1 - extractIntInStr with corrected examples]

```

/**
 * Extracts leading integer. null/empty/non-digit-start -> 0.
 * Examples: null->0, ""->"0", "1234"->1234,
 *           "1234a5"->1234, "9x9"->9, "-1"->0
 */

```

```
public static int extractIntInStr(String str);
```

[Function 2 - parseToken with terminator inclusion clarified]

```

/**
 * Terminator IS INCLUDED: parseToken("abc,def",{' ',';' }) -> "abc,"
 * No terminator -> whole string returned
 */

```

```
public static String parseToken(String str, char[] terminators);
```

[Function 3 - roundSecondToString with exact format]

```

/**

```

```
* Returns "<Math.round(x)> <letter>": 0->"0 S", 60->"1 M",
* 90->"2 M", 3600->"1 H", 86400->"1 D", 604800->"1 W", 31536000->"1 Y"
*/
```

```
public static String roundSecondToString(long second);
```

[Functions 4-5 - dayStr and strToBoolean with truth-table docstrings]

Output only Java code.

[GPT-5.2 RESPONSE] (key tests)

```
// extractIntInStr: all examples used, plus new edge cases derived from spec
@Test
public void testExtractIntInStr_AllDigits_And_StopsAtFirstNonDigit() {
    assertEquals(1234, Subject.StringAlgorithms.extractIntInStr("1234"));
    assertEquals(1234, Subject.StringAlgorithms.extractIntInStr("1234a5")); // fixed!
    assertEquals(7, Subject.StringAlgorithms.extractIntInStr("007x"));
    assertEquals(0, Subject.StringAlgorithms.extractIntInStr("0x"));
    assertEquals(9, Subject.StringAlgorithms.extractIntInStr("9-9"));
}
@Test
public void testExtractIntInStr_NullEmptyNonDigit() {
    assertEquals(0, Subject.StringAlgorithms.extractIntInStr(null));
    assertEquals(0, Subject.StringAlgorithms.extractIntInStr(""));
    assertEquals(0, Subject.StringAlgorithms.extractIntInStr("a"));
    assertEquals(0, Subject.StringAlgorithms.extractIntInStr("-1")); // new!
    assertEquals(0, Subject.StringAlgorithms.extractIntInStr("+1")); // new!
}

// parseToken: terminators correctly included
@Test
public void testParseToken_TerminatorFound_IncludesIt() {
    assertEquals("abc,", parseToken("abc,def", new char[]{' ',''})); // PASSES
    assertEquals("", parseToken(",def", new char[]{' ',''})); // PASSES
    assertEquals("a;", parseToken("a;bc", new char[]{' ',';'}));
    assertEquals(";", parseToken(";tail", new char[]{' ',';'}));
}

// roundSecondToString: exact assertions at boundaries
@Test
public void testRoundSecondToString_Rounding() {
    assertEquals("60 M", roundSecondToString(3599)); // 3599/60~=60
    assertEquals("1 H", roundSecondToString(3600));
    assertEquals("24 H", roundSecondToString(86399)); // just below 1D
    assertEquals("1 D", roundSecondToString(86400));
}

// strToBoolean: priority order tested correctly
```

```
@Test
public void testStrToBoolean_NullStr_PriorityOrder() {
    assertNull(strToBoolean(null, "T", "F", null)); // nullString==null -> null
    assertEquals(TRUE, strToBoolean(null, null, "F", "X")); // trueString==null -> TRUE
    assertEquals(FALSE, strToBoolean(null, "T", null, "X"));
}
```

5.3 Results

Listing 3. Test execution for LLMEnhancedTest

```
$ java -cp ... org.junit.runner.JUnitCore comp5111.assignment.cut.
  LLMEnhancedTest

JUnit version 4.12
.....
Time: 0.028

OK (18 tests)
```

One test initially failed: `strToBoolean(null, null, "F", null)` was expected to return `TRUE` (model reasoned: `trueString==null ⇒ TRUE`) but the implementation checks `nullString==null` first and returns `null`. After correcting the input to `strToBoolean(null, null, "F", "X")` (non-`null nullString`), all 18 tests pass. This actually illustrates that even an explicit priority table in the docstring isn’t quite enough when two sentinels are simultaneously `null`—the model still need to trace the conditional chain more carefully.

5.4 Before/After Comparison

Test suite	Tests	Failures	Assertion style	New cases
Simple (5 funcs)	~12	3	Fuzzy / wrong value	—
Enhanced (5 funcs)	18	0	Exact assertEquals	"-1"->0, rounding, full boundaries

Table 5. Same 5 functions: simple vs. enhanced specification

The improvements are pretty clear:

- `parseToken`: 2 failures → 0, just by adding the word “INCLUDED” to the docstring.
- `extractIntInStr`: 1 failure → 0; also gained tests for “-1” and “+1”.
- `roundSecondToString`: fuzzy `contains()` → exact boundary assertions (3599s → “60 M” was tested for the first time).
- `strToBoolean`: priority-order tests now generated correctly from the truth table.

Overall, the most impactful single change was fixing wrong examples (eliminates failures immediately) and the most automatable change is having an LLM generate verified examples from the function body.